

laSalle Health Center

Sistema de Soporte Hospitalario con IA, Big Data y Deep Learning

Autores: Ariadna Pascual y Pol Ballarín



Docker Compose



Inteligencia Artificial



Big Data



Dashboard Clínico

 **Guion:** Presentarse ante el tribunal. Introducir el proyecto: una plataforma integral que une pipelines de ingesta, bases de datos heterogéneas (SQL y NoSQL), arquitecturas de inferencia de IA y monitorización para asistir en la toma de decisiones clínicas.

Problema y Objetivos

⚠ El Reto Hospitalario

La simulación de un entorno clínico real implica gestionar un flujo continuo y masivo de datos heterogéneos: datos clínicos estructurados, pacientes, radiografías pesadas, logs del sistema y eventos en tiempo real. Existe una brecha crítica para consolidar esta información y usarla activamente en soporte clínico.

🎯 Objetivos del Sistema

Diseñar un sistema distribuido que permita:

- ✓ Validar e ingerir datos en calidad y volumen.
- ✓ Clasificar pacientes mediante algoritmos de triaje.
- ✓ Procesar y diagnosticar radiografías automáticamente.
- ✓ Almacenar y monitorizar logs, eventos y alertas.

🎙 **Guion:** Explicar el contexto: no solo queríamos hacer modelos aislados, sino un ecosistema que simule un hospital real donde los datos clínicos y no estructurados convergen en bases de datos seguras y paneles visuales rápidos.

Arquitectura Global



1. Ingesta y API

Interfaz web y API REST en FastAPI que actúan como puerta de entrada para formularios de pacientes individuales, carga masiva de CSV clínicos y almacenamiento de radiografías en MinIO.



2. Pipeline e IA

Orquestador que desencadena el procesamiento. MinIO actúa como Landing Zone. Dask gestiona la paralelización hacia PostgreSQL (datos relacionales) y MongoDB (predicciones, logs y alertas).



3. Salidas y Alertas

Dashboard clínico en tiempo real (Streamlit), exportación de informes en PDF e infraestructura de logs centralizados mediante Promtail, Loki y automatización de alertas.

 **Demo:** "Para entender cómo interactúan estos contenedores, vamos a ver nuestro archivo `docker-compose.yml`". **Cambio a VS Code:** Mostrar el compose, la red interna común, los volúmenes persistentes y cómo se levantan FastAPI, Streamlit, PostgreSQL, MongoDB, MinIO y Loki.

Estructura del Repositorio

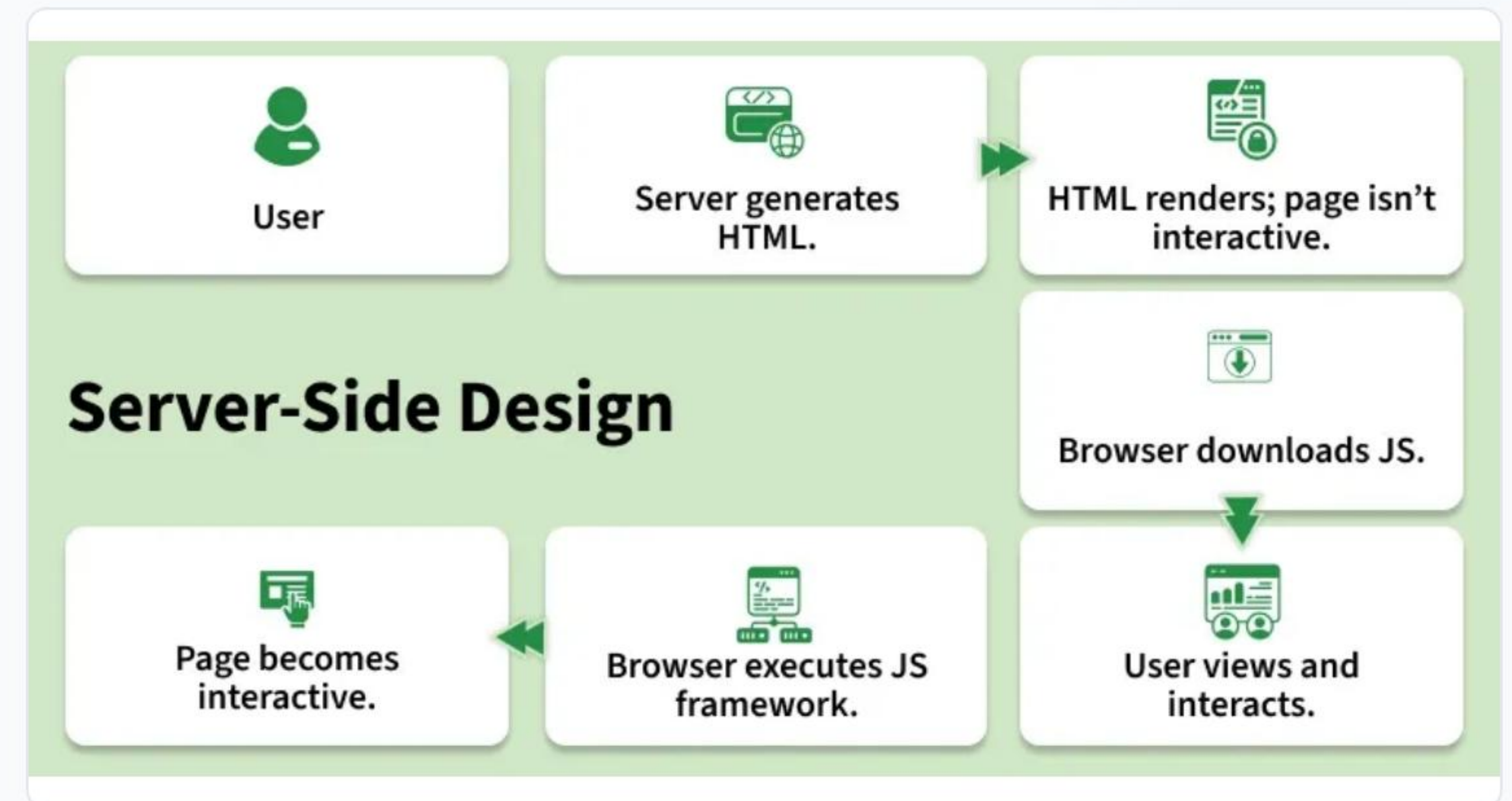
- 📁 **services/api:** Puntos de enlace REST en FastAPI para interacción con clientes y modelos.
- 📁 **services/pipeline:** Ingesta estructurada Batch/Stream con Dask y validaciones de datos.
- 📁 **services/ml-inference:** Pipelines de Deep Learning para clasificación de rayos X en PyTorch.
- 📁 **notebooks:** Entornos de experimentación y validación (de 00_ a 05_).
- 📁 **services/dashboard:** Panel médico con Streamlit para visualizar el estado global.
- 📁 **services/ml-triage:** Clasificador tabular basado en Machine Learning (triaje y sospechas).
- 📁 **services/automation:** Escáner y despachador de alertas y reportes desatendidos.
- 📁 **monitoring:** Configuraciones del stack Loki / Promtail para centralizar telemetría.

🎤 **Demo:** "Esta estructura limpia garantiza el desacoplamiento de componentes". **Cambio a VS Code:** Mostrar la carpeta raíz del proyecto, abriendo brevemente los directorios `services/` y `notebooks/` para demostrar la organización real del repositorio.

Pipeline Big Data

El flujo de datos está diseñado para escalar de forma masiva y segura:

-  **Ingesta Batch:** Los CSV clínicos se depositan inicialmente en un bucket de MinIO (Landing Zone / Raw).
-  **Calidad de Datos:** Reglas estrictas validan tipos, nulos y rangos lógicos antes de persistir los registros.
-  **Dask Engines:** Procesamiento concurrente de registros que distribuye la carga de escritura.
-  **Destinos Híbridos:** Pacientes e ingresos van a PostgreSQL (Sólido ACID), y eventos, alertas y logs JSON a MongoDB.



 **Demo:** "Hemos documentado toda la lógica del flujo". **Notebook sugerido:** Abrir el notebook `01_pipeline_bigdata.ipynb` y enseñar cómo Dask carga un volumen simulado y realiza transformaciones rápidas guardando de manera paralela en base de datos.

Modelos Tabulares: Triage

3

Prioridades de Atención

Clasificación de Gravedad

El módulo `ml-triage` utiliza algoritmos supervisados para clasificar el nivel de urgencia basándose en los síntomas auto-reportados y constantes vitales:

- ⚠ **Clases de Triage:** Alta, Media y Baja prioridad.
- 🏥 **Sospecha Clínica:** Diagnóstico preliminar de patologías base.
- 💚 **Criterio de Confianza:** Si la probabilidad máxima del modelo es baja, se deriva obligatoriamente a revisión manual para evitar sesgos peligrosos.

🎧 **Demo:** "Para entrenar este modelo usamos un XGBoost calibrado". **Notebook/Dashboard:** Mostrar brevemente `02_ml_triage_tabular.ipynb` para ver la importancia de variables o cargar el Dashboard y rellenar un formulario rápido para ver cómo responde con "Triage: ALTA".

Deep Learning: Radiografías

El reto consiste en implementar un clasificador automático capaz de distinguir entre tres estados pulmonares críticos:

-  **Dataset:** "COVID-19 Radiography Dataset", excluyendo máscaras y limpiando la jerarquía para dejar únicamente imágenes en crudo.
-  **Entrenamiento:** Proceso riguroso que evalúa arquitecturas con transferencia de aprendizaje.
-  **Modelos Comparados:** Red neuronal convolucional personalizada (Simple CNN), ResNet18 y EfficientNet-B0.

 Chest X-Ray Clinical Radiography analysis

 **Demo:** "Hablemos de la preparación de los datos en PyTorch". **Notebook sugerido:** Abrir `03_ml_inference_radiografias.ipynb` y mostrar el bloque donde se definen las transformaciones y aumentos de imagen aplicadas a las placas de rayos X.

Resultados de Clasificación

Modelo Evaluado	Accuracy Global	F1-Macro	Recall COVID-19	Recall Neumonía	Criterio Técnico / Clínico
EfficientNet-B0	0.9600	0.9601	0.9800	0.9333	Ganador absoluto: Mayor balance clínico y menor tasa de falsos negativos.
ResNet18	0.8933	0.8929	0.9667	0.8333	Competitivo pero con fallos críticos en la detección de Neumonía común.
Simple CNN	0.7022	0.7013	0.7000	0.7733	Línea base insuficiente. Demasiados errores en diagnóstico diferencial.

 **Guion:** Analizar la tabla ante el tribunal. Resaltar que no solo nos fijamos en la Accuracy global, sino en el Recall de COVID-19 y Neumonía; en medicina, un falso negativo (enviar a un enfermo a casa) es un error crítico. EfficientNet-B0 ofrece las mejores garantías.

Criterio Clínico y Overfitting

Control de Overfitting

Para asegurar que el modelo generaliza correctamente, implementamos separación estricta (Train/Val/Test), Data Augmentation (rotación, brillo) y guardamos la telemetría en `history.csv` monitorizando el F1 macro:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

Limitaciones del Modelo

Es indispensable mencionar ante el tribunal:

-  **No entrenamiento interactivo:** Al subir una imagen, el backend hace solo inferencia estática; no se reentrena en caliente.
-  **Validación externa:** Al carecer de datasets de otros hospitales, el modelo puede heredar sesgos específicos del dataset original.

 **Guion:** Explicar que el uso del F1-Macro evita que las clases mayoritarias oculten el mal rendimiento en clases críticas. Detallar el uso de la fórmula de F1 en el entrenamiento de la red convolucional.

Monitorización y Alertas



Loki y Promtail

Todos los logs de los contenedores Docker se centralizan activamente mediante Promtail y se envían a Loki, permitiendo auditorías técnicas en tiempo real.



system_events

Colección específica en MongoDB que registra el estado del pipeline, auditorías de ingesta, ejecuciones de Dask y errores lógicos del negocio.



Automation Engine

Servicio desatendido que escanea errores/warnings de forma recurrente y escribe alertas en MongoDB, deduplicando avisos clínicos críticos.

 **Demo:** "La telemetría no es solo para el equipo técnico, sino que nutre el sistema de alertas del hospital". **Notebook sugerido:** Abrir el notebook `04_monitoring_automation.ipynb` y enseñar las consultas para visualizar eventos del sistema y alertas disparadas.

Dashboard Clínico

El Centro de Control Médico

Streamlit unifica la interacción del personal sanitario con la arquitectura del sistema, ofreciendo cuatro flujos de trabajo en producción:

-  **Triage Individual:** Formulario rápido asistido por IA.
-  **Carga Batch:** Carga de CSV para procesamiento paralelo.
-  **Rayos X:** Arrastrar y predecir patologías respiratorias.
-  **Estado del Sistema:** Visor de logs y alertas activas.

CLINICAL ANALYSIS

Patients Demographics

Key Trends

Treatment & Cost



Rooms / Bedspace

398

▼ -3% Vs 2021



Avg Billing Amount

\$25,350

▼ -2.7% Vs 2021



Doctors

2,666

▲ 2.1% Vs 2021



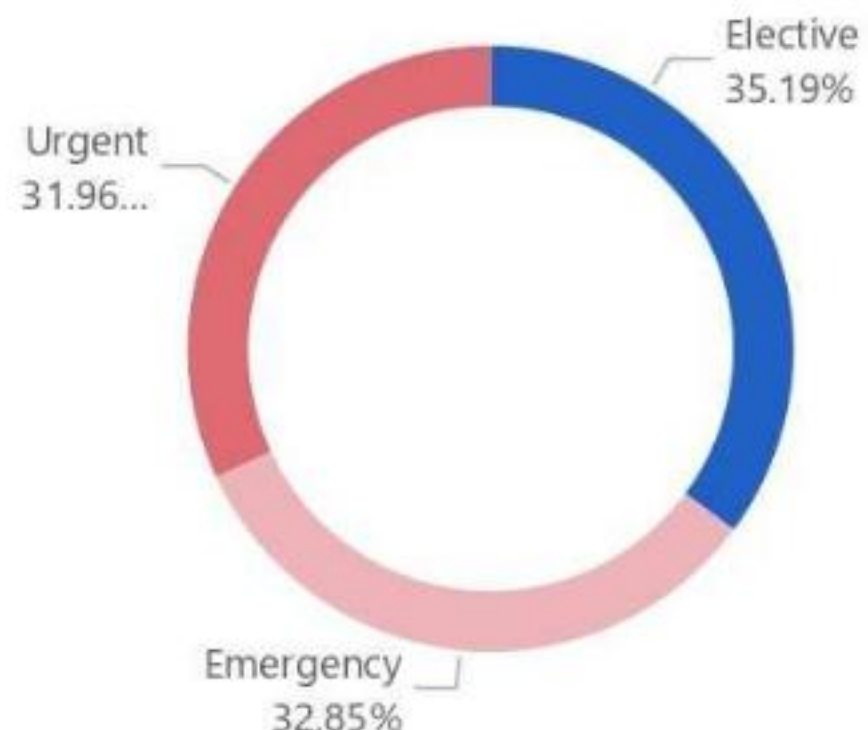
Avg Billing Amount

1

▲ 2.1% Vs 2021

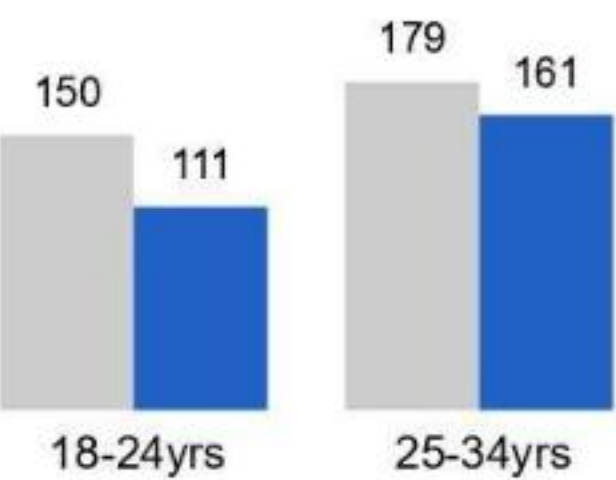


Admitted Patients by Admission Type



Agegroup

Gender ● Female ● Male

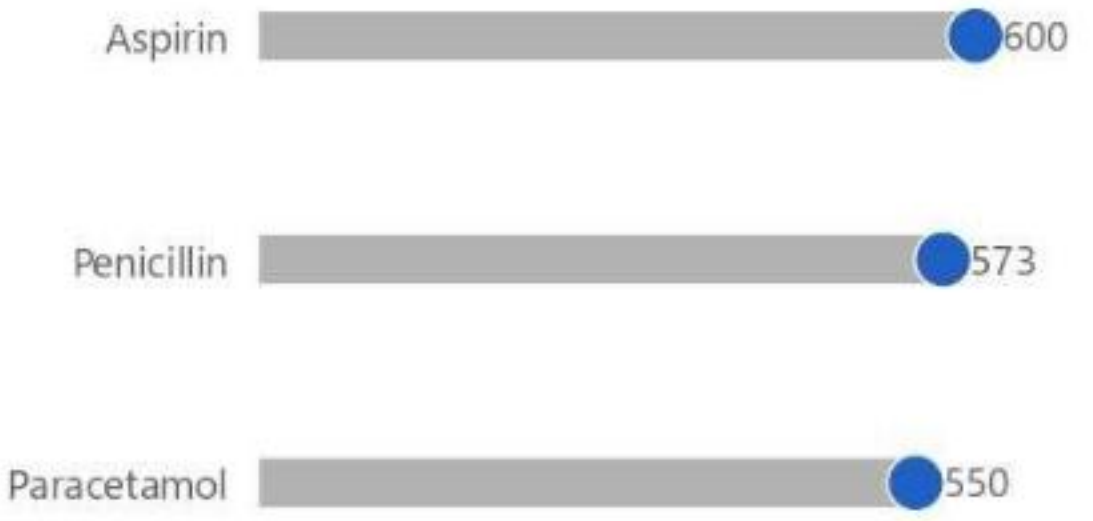


lood where
ainst ly too
k of and
is.



Recommendations

Recommended Medications



Hospital	Arthritis
Cedars-Sinai Medical Center	53
Cleveland Clinic	58
Houston Methodist Hospital	445
Johns Hopkins Hospital	222

 **Demo en vivo:** Abrir el navegador en <http://localhost:8502>. Mostrar el formulario de triaje, cargar un CSV de prueba y arrastrar una radiografía (p. ej. de un paciente COVID). Explicar que Streamlit llama transparentemente a la API y a los microservicios ml-inference.

Notebooks de Evidencia

-  **00_overview_sistema.ipynb:** Diagnóstico de conectividad con puertos de Postgres, MongoDB, Loki y MinIO.
-  **02_ml_triage_tabular.ipynb:** Entrenamiento, calibración de probabilidad e importancia de variables en triaje.
-  **04_monitoring_automation.ipynb:** Demostración de cómo se recuperan y agrupan los logs desde Loki y alertas de Mongo.
-  **01_pipeline_bigdata.ipynb:** Pruebas unitarias de ingesta masiva de CSV clínicos y validación de tipos.
-  **03_ml_inference_radiografias.ipynb:** Curvas de aprendizaje, carga de EfficientNet y predicción de rayos X.
-  **05_dashboard_api_demo.ipynb:** Consumo e interacción directa con los endpoints de la API documentada en FastAPI.

 **Guion:** Aclarar que en este proyecto los notebooks no guardan código duplicado de producción, sino que sirven como bitácora científica interactiva, asegurando la reproducibilidad de todos los experimentos realizados.

Ética, Privacidad y Límites

Protección de Datos

Dado que tratamos con datos médicos sensibles:

-  **Pseudonimización:** Cifrado de identificadores únicos en el pipeline.
-  **Seguridad:** Archivos .env fuera de control de versiones.
-  **Privacidad:** Los datasets originales nunca se suben a repositorios públicos.

Apoyo vs Diagnóstico

Nuestra plataforma está concebida estrictamente como un **Sistema de Soporte Hospitalario**:

-  No reemplaza el criterio médico humano bajo ninguna circunstancia.
-  No está homologado como dispositivo o software médico regulado (CE/FDA).

 **Guion:** El tribunal valora positivamente la autocrítica ética. Explicar cómo el sistema previene el sesgo de automatización (médicos aceptando a ciegas lo que dice la IA) y los límites de la pseudonimización del pipeline.

Conclusiones del Proyecto

- ✓ **Entorno Containerizado:** Ecosistema completamente reproducible y escalable mediante Docker Compose.
- ✓ **Criterio Clínico IA:** Comparación real de arquitecturas que sitúa la seguridad del paciente sobre el accuracy simple.
- ✓ **Dashboard Clínico:** Interfaz unificada que valida la utilidad del backend en un escenario operativo simulado.
- ✓ **Pipeline Híbrido:** Almacenamiento eficiente que combina la solidez de SQL con la flexibilidad JSON.
- ✓ **Monitorización Activa:** Infraestructura robusta para detectar caídas de servicio o alertas clínicas rápido.
- ✓ **Documentación Completa:** Código limpio, notebooks autoexplicativos y guías detalladas de arquitectura.

🎤 **Guion:** Terminar con un mensaje de cierre contundente: el proyecto demuestra cómo la IA y el Big Data pueden integrarse hoy mismo de forma modular para optimizar las operaciones de un hospital real, sirviendo de copiloto al profesional de salud.

Plan de Demo en Vivo

-  **Paso 1: Estado del Docker:** Comprobar contenedores corriendo (docker compose ps) desde VS Code.
-  **Paso 3: Evidencias en Notebooks:** Abrir el Notebook 03 de Deep Learning y repasar las métricas obtenidas.
-  **Paso 5: Carga de Radiografía:** Cargar placa en el portal y esperar inferencia de EfficientNet.
-  **Paso 2: Arquitectura del Repo:** Demostrar modularidad técnica en el árbol de carpetas del IDE.
-  **Paso 4: Formulario de Triage:** Probar un caso grave en el dashboard (localhost:8502).
-  **Paso 6: Monitorización:** Visualizar las alertas automatizadas creadas en MongoDB.

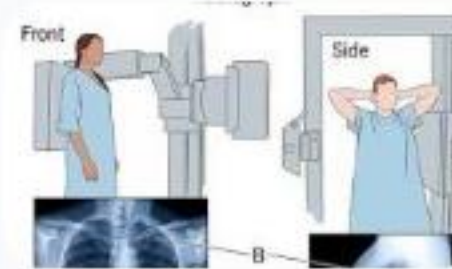
 **Demo / Q&A:** Dejar esta diapositiva proyectada mientras realizas la demostración en vivo. Sirve como guion visual para vosotros y demuestra al tribunal el orden y rigor de vuestra presentación técnica. Abrir turno de preguntas.

Image Sources



https://media.geeksforgeeks.org/wp-content/uploads/20251227172124991832/server_side_design.webp

Source: www.geeksforgeeks.org



<https://my.clevelandclinic.org/-/scassets/images/org/health/articles/10228-chest-x-ray>

Source: my.clevelandclinic.org



<https://community.fabric.microsoft.com/t5/image/serverpage/image-id/1288877i71AA4AA61F347690?v=v2>

Source: community.fabric.microsoft.com